Exercise Sheet 07

The Toolbox

House rules.

- One file per exercise, in `my_work/`. Exercise 1 is done in the terminal, no file.
- Run everything **from the lesson folder**, with the virtual environment active (your prompt starts with `(.venv)`).
- Type the **Given** lines first, then write the rest. Compare with the **expected output**.
- Hand in: commit and push to `lesson-07`, open a pull request. Check `git status` first: `.venv/` must NOT be in it.

Exercise 1 A toolbox of your own

10 min · terminal only

> **From Mo (IT):** *"Before you install anything: a virtual environment. It's a private toolbox for this project, so whatever you install here can't break anything else. Also I don't have to hear about it."*

IN THE TERMINAL, INSIDE THE LESSON FOLDER

```
python -m venv .venv            # create the toolbox (once per project)
.venv\Scripts\activate          # open it: the prompt now starts with (.venv)
pip install -r requirements.txt # install what's on the list
pip list                        # what's in the box?
python -c "import rich; print('rich is ready')"
```

(a) Run the five commands. `pip list` shows `rich` and a few friends it brought along. Version numbers will differ from your neighbour's; that's fine.

(b) Open `requirements.txt` in VS Code and read it. That one line is the whole shopping list.

(c) Run `git status`. If `.venv/` shows up as untracked, tell Lars: the `.gitignore` should hide it.

EXPECTED OUTPUT (LAST COMMAND)

```
rich is ready
```

Hint: "Running scripts is disabled"? Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then activate again. New terminal = activate again. VS Code usually does it for you once it has noticed the `.venv` folder.

Exercise 2 Borrow from the standard library

8 min · `ex2_maths.py`

> **From Gerald (MD):** *"Siobhan says I'm not allowed to say 'average' any more without saying which one. Also, if I split eight people into teams of three, how many teams is that? Mo laughed."*

GIVEN

```
import math
import statistics

salaries = [24950, 26550, 29000, 31750, 37650, 41900, 45200, 68750]
```

(a) Print the mean (2 decimals), the median, the standard deviation (`statistics.stdev`, 2 decimals) and the range (max minus min).

(b) Print how many teams of 3 you need for 8 people (`math.ceil`) and how many *full* teams you get (`math.floor`).

(c) Print the total salary bill rounded *up* to the nearest £1,000, and the square root of 1952 to 3 decimals.

EXPECTED OUTPUT

```
Mean:    38218.75
Median:  34700.0
Std dev: 14311.93
Range:   43800
Teams of 3: 3
Full teams of 3: 2
Budget rounded up to the nearest £1,000: 306000
Square root of 1952: 44.181
```

Hint: Nobody memorises the standard library. Type `python`, then `import statistics`, then `help(statistics.stdev)`. `q` gets you out of the help page. "Round up to the nearest 1,000" is `ceil(x / 1000) * 1000`.

Exercise 3 Dates without tears

12 min · `ex3_dates.py`

> **From Priya (Payroll):** *"Length of service, in years, as of the snapshot date, 5 April. In Excel this is a DATEDIF and a prayer. And when's payday? It's the 25th. I know when payday is. I want the script to know."*

GIVEN

```
from datetime import date, timedelta

today = date(2026, 4, 5)
start_dates = {"Priya": "2017-03-13", "Mo": "2021-01-04", "Siobhan": "2024-11-28"}
```

(a) For each person: turn the text into a date with `date.fromisoformat()`, work out the days since then (`(today - start).days`) and the years (divide by 365.25). Print the name in 8, the start date as "13 Mar 2017" and the weekday in brackets, then the years with 1 decimal.

(b) Payday is the 25th. Build this month's payday; if it's already gone, use next month's. Print it in ISO format and how many days away it is.

(c) The report deadline is 90 days from today. Print it as dd/mm/yyyy.

EXPECTED OUTPUT

```
Priya    started 13 Mar 2017 (Monday), 9.1 years ago
Mo       started 04 Jan 2021 (Monday), 5.2 years ago
Siobhan  started 28 Nov 2024 (Thursday), 1.3 years ago
Next payday: 2026-04-25 in 20 days
Report deadline: 04/07/2026
```

Hint: Dates format inside f-strings: `f"{start:%d %b %Y}"`, `f"{start:%A}"`. Codes: `%d` day, `%m` month number, `%b` short month name, `%Y` year, `%A` weekday. `today + timedelta(days=90)` does the calendar maths for you.

Exercise 4 Your own module

12 min · `paytools.py` + `ex4_use_paytools.py`

> **From Mo:** *"You've now typed `monthly salary = annual / 12` into six different files. The seventh time, put it in a module and import it. Future Jen will send a thank-you card."*

GIVEN (IN `MY_WORK/PAYTOOLS.PY`)

```
"""Pay calculations for Brightwell Umbrellas. Import me, don't copy me."""
```

(a) In `paytools.py`, write four functions with docstrings: `monthly(annual_salary)` (2 decimals), `hourly(annual_salary, weekly_hours=37.5)` (52 weeks, 2 decimals), `pay_gap_percent(male_hourly, female_hourly)` (UK definition, 1 decimal) and `money(amount)` (£, commas, 2 decimals). No `print` anywhere in this file.

(b) In `ex4_use_paytools.py`: `import paytools`, then for Priya (32500), Mo (41900) and Siobhan (37650) print the monthly pay (right-aligned in 10) and the hourly rate, both as money. Then print Mo's hourly rate at 22.5 hours, and the gap for £20.00 vs £17.40.

EXPECTED OUTPUT

```
Priya     £2,708.33 a month, £16.67 an hour
Mo        £3,491.67 a month, £21.49 an hour
Siobhan   £3,137.50 a month, £19.31 an hour
Part-time Mo: 35.81
Gap: 13.0 %
```

Hint: Both files must be in `my_work/`, and you run `python my_work/ex4_use_paytools.py`: Python looks for modules next to the file it's running. In the REPL, `help(paytools)` shows your docstrings. That's what they're for.

Exercise 5 Biscuit's raffle

6 min · `ex5_raffle.py`

> **From Siobhan (Reward):** *"Umbrella of the Month, the Christmas raffle and the tea rota are all currently decided by Gerald 'at random'. Everyone has noticed it's always Sales. Make it actually random. But the same random every time, so nobody can argue."*

GIVEN

```
import random

random.seed(1952)
names = ["Priya", "Mo", "Siobhan", "Amara", "Ben", "Chloe", "Dev", "Freya"]
```

(a) Print one winner with `random.choice`, then three different raffle winners with `random.sample`, joined with ", ".

(b) `random.shuffle` the list and print the first four as a tea rota, joined with " -> ".

(c) Roll a die with `random.randint(1, 6)` for the length of Biscuit's walk.

EXPECTED OUTPUT

```
Umbrella of the month goes to: Chloe
Christmas raffle winners: Chloe, Dev, Ben
Tea rota: Chloe -> Mo -> Ben -> Freya
Dice for Biscuit's walk: 6
```

Hint: The seed makes "random" repeatable. Keep the calls in exactly this order, or you'll get different (equally random) names. Remove the seed line and run twice to see the difference.

Exercise 6 Something you didn't write

8 min · `ex6_rich_table.py`

> **From Gerald:** *"Can the terminal output look less like 1983? Mo says there's 'a package for that'. There's always a package for that."*

GIVEN

```
from rich.console import Console
from rich.table import Table

staff = [("EMP1092", "Max Robinson", "Sales", 41900),
         ("EMP1178", "Imogen Shaw", "Marketing", 26550),
         ("EMP1105", "Amara Roberts", "Sales", 24950)]
```

(a) Make a `Table(title="Brightwell staff")` with columns ID, Name, Department and Salary (right-justified). Add one row per person, salary as £ with commas.

(b) Print it with `Console(width=60).print(table)`, then a second line with "3 rows" in bold green: `"[bold green]3 rows[/bold green], no MACRO-9000 involved"`.

EXPECTED OUTPUT (THE BOX LINES MAY LOOK SLIGHTLY DIFFERENT IN YOUR TERMINAL)

```
                Brightwell staff
┏━━━━━━━━━┳━━━━━━━━━━━━━━┳━━━━━━━━━━━━┳━━━━━━━━━━┓
┃ ID      ┃ Name         ┃ Department ┃   Salary ┃
┡━━━━━━━━━╇━━━━━━━━━━━━━━╇━━━━━━━━━━━━╇━━━━━━━━━━┩
│ EMP1092 │ Max Robinson │ Sales      │ £41,900  │
│ EMP1178 │ Imogen Shaw  │ Marketing  │ £26,550  │
│ EMP1105 │ Amara Roberts│ Sales      │ £24,950  │
└─────────┴──────────────┴────────────┴──────────┘
3 rows, no MACRO-9000 involved
```

Hint: You've never seen `rich` before. That's the exercise: read the *Given*, guess from the names (`table.add_column(...)`, `table.add_row(...)`), and when stuck, search "rich table python". Reading other people's documentation is most of programming.

Exercise 7 The main guard STRETCH

8 min · `paytools.py` + `ex7_main_guard.py`

> **From Mo:** *"One more trick. A module should be quiet when imported and chatty when run on its own. There's a line for that. You've seen it before, in lesson 00, and nodded politely."*

GIVEN (ADD TO THE END OF `PAYTOOLS.PY`)

```
if __name__ == "__main__":
    print("paytools self-test")
```

(a) Under that line, print a one-line check for each of the four functions, in the format shown below.

(b) Run `python my_work/paytools.py`: you get the self-test.

(c) Write `ex7_main_guard.py`: `from paytools import monthly, money`, then print the two lines below. Run it: no self-test appears.

EXPECTED OUTPUT OF `PYTHON MY_WORK/PAYTOOLS.PY`

```
paytools self-test
monthly(32500)        -> 2708.33
hourly(32500)         -> 16.67
hourly(32500, 22.5)   -> 27.78
pay_gap_percent(20, 17.4) -> 13.0
money(2708.333)       -> £2,708.33
```

EXPECTED OUTPUT OF `PYTHON MY_WORK/EX7_MAIN_GUARD.PY`

```
Importing paytools printed nothing above this line.
Monthly pay for £45,200: £3,766.67
```

Hint: `__name__` is `"__main__"` only in the file you ran directly. Inside an imported module it's the module's name, so the block is skipped.

Bug hunt Dave's Leftovers #7

15 min · `bughunt/toolbox_bug.py`

Dave's team planner, written the week he discovered `import`. There are **three** bugs: one crashes on the very first line, one rounds the wrong way so one team never gets a desk, and one prints something technically true but not what was asked.

EXPECTED OUTPUT ONCE FIXED

```
Staff: 9
Average salary: £31,644.44
Teams of 4 needed: 3
Days since Dave left: 2410
```

Commit each fix separately, with a message explaining what was wrong.